МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №2

з курсу: Безпека інформаційних систем

виконав студент 4 курсу
групи ДА-11
Сидько Н.С

КИЇВ
2024

Завдання

Розробка програми генератора великих простих чисел (ВПЧ) для шифрування та обчислення ключів за схемою RSA за декількома десятками десяткових знаків. Для створення таких простих чисел можна використовувати формули відповідно до тесту Рабіна чи інші методи, але для перевірки властивостей сформованих кандидатів в прості числа необхідно використовувати малу теорему Ферма.

Виконання завдання

Напишемо код на мові Python через свою гнучкість і простоту у використанні, також здатність працювати з великими числами без обмежень розрядності.

Тест Ферма

Тест Ферма базується на малій теоремі Ферма, яка стверджує, що якщо p є простим числом і a є цілим числом, яке не ділиться на p, то виконується: $a^{p-1} \equiv 1 \pmod{p}$, якщо піднести a до степеня p-1 і обчислити залишок від ділення на p, результат буде 1.

Вибирається випадкове число a в діапазоні від 2 до n-2, де n — число, яке потрібно перевірити. Обчислюється $a^{n-1} \bmod n$. (Якщо результат не дорівнює 1, то n не є простим. Якщо результат дорівнює 1 для кількох вибраних a, то n вважається простим з певною ймовірністю.) Процедура повторюється k разів, де k — кількість перевірок

```python
def is_fermat_prime(n, k=5):  1 usage
    if n < 2:
        return False

    for _ in range(k):
        a = random.randint( a: 2, n - 2)
        if pow(a, n - 1, n) != 1:
            return False

    return True
```

Якщо число n менше 2, то воно не може бути простим, тому повертається False. Виконується k перевірок. На кожному кроці обирається випадкове число a в діапазоні від 2 до n-2. Обчислюється a^(n-1) mod n за допомогою функції pow(). Якщо результат не дорівнює 1, то число n не є простим, і повертається False.

Тест Рабіна-Міллера

Розкладається $n-1$ як $2^s \cdot d$ де d — непарне число (наприклад, якщо $n-1 = 24$ то $24=2^3 \cdot 3$, тому $s=3$ $d=3$.

Перевірка основ повторюється k разів для різних випадкових значень a:

- Вибирається випадкове число a в діапазоні від 2 до n-2.
- Обчислюється $x=a^d \bmod n$.
- Якщо $x=1$ або $x= n-1$число вважається пройденим для цієї перевірки, і переходять до наступного значення a.
  Якщо ні, то перевіряється $x^{2^r} \bmod n$ для r від 0 до $s-1$ (Якщо будь-яке з цих обчислень дорівнює $n-1$, число вважається пройденим для поточного значення a.
  Якщо жодне з обчислень не дає результату $n-1$ число є складеним)

```python
def is_rabin_miller_prime(n, k=5):  1 usage
    if n < 2:
        return False
    if n == 2 or n == 3:
        return True
    if n % 2 == 0:
        return False

    s, d = 0, n - 1
    while d % 2 == 0:
        d //= 2
        s += 1

    for _ in range(k):
        a = random.randint( a: 2, n - 2)
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue

        for _ in range(s - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False

    return True
```

- Перевіряється, чи n менше 2 або є парним. Якщо так, число не є простим.
- Розкладається $n-1$ у вигляді $2^S \cdot d$
- Для k випадкових значень a обчислюється $a^d \bmod n$
- Перевіряється, чи дорівнює результат 1 або $n-1$, або чи $x^{2^r} \bmod n = n-1$
- Якщо хоча б одна перевірка не пройшла, число вважається складеним.

Функції генерації великих чисел, та використання тестування.

```python
def generate_large_numbers(count):  1 usage
    numbers = []
    for _ in range(count):
        num_digits = random.randint( a: 45,  b: 55)
        candidate = random.randint(10 ** (num_digits - 1), 10 ** num_digits - 1)
        numbers.append(candidate)
    return numbers

def check_prime(candidate):  1 usage
    fermat_result = is_fermat_prime(candidate)
    rabin_miller_result = is_rabin_miller_prime(candidate)

    if fermat_result and rabin_miller_result:
        return "Є простим"
    else:
        return "Не є простим"
```

Результат роботи коду

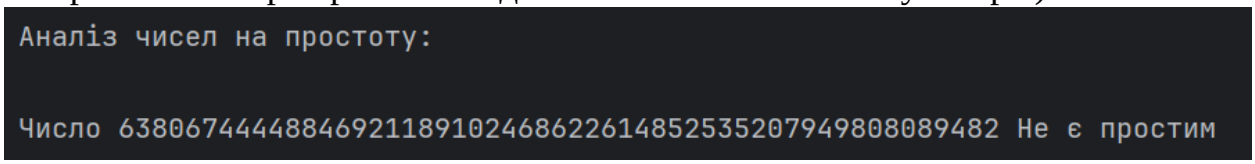
```
Згенеровані числа:
Число: 6380674444884692118910246862261485253520794980808948 2
Число: 4987418398221245391396766485773468065252021173637
Число: 3124445936305657758696580479322070289428136258895521 0
Число: 2067870112583512970703428851377238870570531021585454
Число: 7488384807080360250112642630053124246885207307197709
Число: 7297262868842550991170609511487541199717259100378453 28
Число: 5088716164632130834387978262888315512486754237221 57
Число: 6423146506216550751439372250277434918149474520 67
Число: 1743079662903457905094795000027115280189914082366086
Число: 2948117242619156826674921938826960795095900865627 12

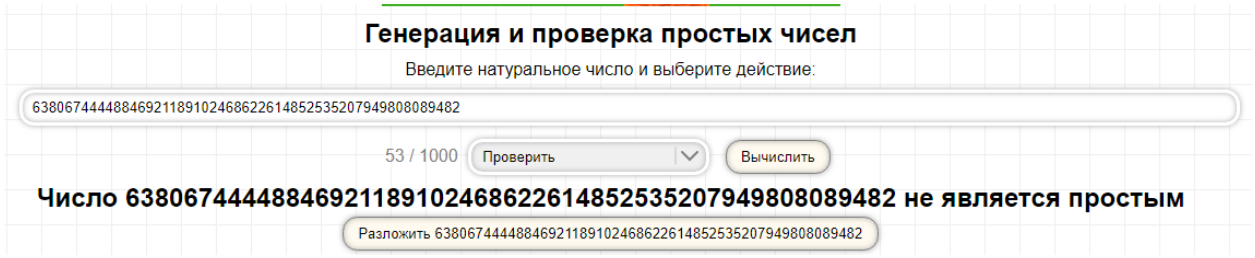
Аналіз чисел на простоту:

Число 6380674444884692118910246862261485253520794980808948 2 Не є простим
Число 4987418398221245391396766485773468065252021173637 Є простим
Число 3124445936305657758696580479322070289428136258895521 0 Не є простим
Число 2067870112583512970703428851377238870570531021585454 Не є простим
Число 7488384807080360250112642630053124246885207307197709 Не є простим
Число 7297262868842550991170609511487541199717259100378453 28 Не є простим
Число 5088716164632130834387978262888315512486754237221 57 Не є простим
Число 6423146506216550751439372250277434918149474520 67 Не є простим
Число 1743079662903457905094795000027115280189914082366086 Не є простим
Число 2948117242619156826674921938826960795095900865627 12 Не є простим

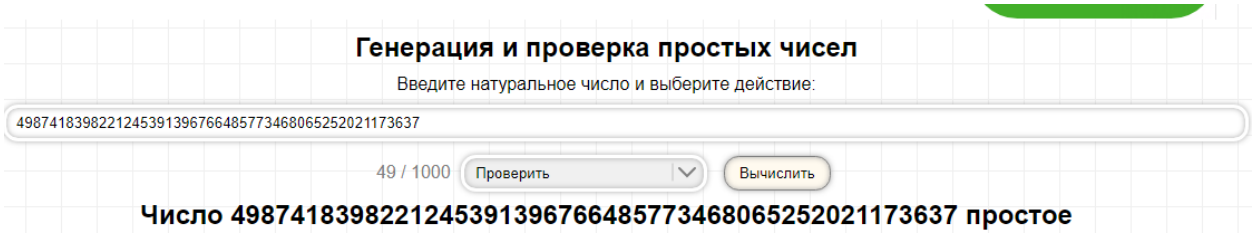
Process finished with exit code 0
```

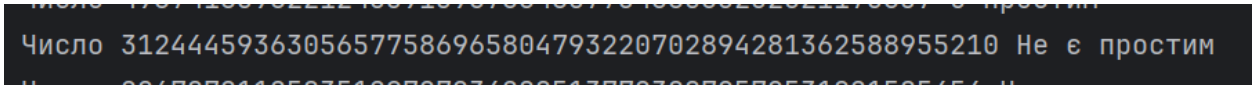
Перевірка на корректність результатів (візьмемо 1 просте число в рисунку та 2 не простих та перевіримо їх за допомогою онлайн калькулятора)

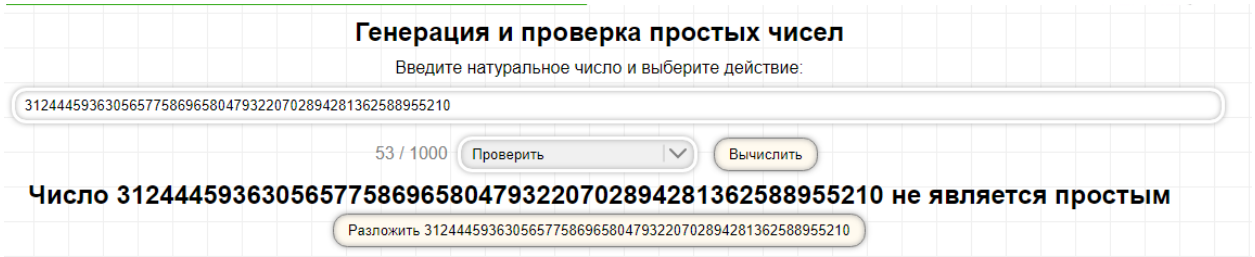












Висновок

У ході виконання лабораторної роботи №2 була написана програма на мові Python для виявлення простих чисел за допомогою тестів Ферма та Рабіна. Тестуючи 10 видпадково згенерованих чисел ми перевіряли їх на простоту за допомогою обох тестів для більш точного результату. Для перевірки корректності роботи програми числа повторно перевірялись на простоту за доппомогою онлайн калькулятора простих чисел

Лістинг коду

```
import random

def is_fermat_prime(n, k=5):
    if n < 2:
        return False

    for _ in range(k):
        a = random.randint(2, n - 2)
        if pow(a, n - 1, n) != 1:
            return False

    return True

def is_rabin_miller_prime(n, k=5):
    if n < 2:
        return False
    if n == 2 or n == 3:
        return True
    if n % 2 == 0:
        return False

    s, d = 0, n - 1
    while d % 2 == 0:
        d //= 2
        s += 1

    for _ in range(k):
        a = random.randint(2, n - 2)
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue

        for _ in range(s - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False

    return True

def generate_large_numbers(count):
    numbers = []
    for _ in range(count):
        num_digits = random.randint(45, 55)
        candidate = random.randint(10 ** (num_digits - 1), 10 ** num_digits - 1)
        numbers.append(candidate)
    return numbers

def check_prime(candidate):
    fermat_result = is_fermat_prime(candidate)
    rabin_miller_result = is_rabin_miller_prime(candidate)

    if fermat_result and rabin_miller_result:
        return "Є простим"
    else:
        return "Не є простим"

def find_primes():
    while True:
        numbers_to_test = generate_large_numbers(10)
```

```
        print("\nЗгенеровані числа:")
        for number in numbers_to_test:
            print(f"Число: {number}")

        print("\nАналіз чисел на простоту:\n")

        prime_found = False
        for number in numbers_to_test:
            result = check_prime(number)
            print(f"Число {number} {result}")
            if result == "Є простим":
                prime_found = True

        if prime_found:
            break
        else:
            print("\nСеред згенерованих чисел немає простих. Генерація нових чисел...\n")

find_primes()
```